arbitrary URLs that don’t map to file paths. They map to database entries instead. You will need a route template that matches anything, so the endpoint that is selected can take care of looking up the URL in the database and displaying the related content. The routing system provides a *catchall parameter* to cater to this scenario.

A catchall parameter is declared by prefixing the name with one or two asterisks—for example, `{*cityName}` or `{**cityName}`. Catchall parameters match everything from the parameter position in the URL to the end, so it doesn’t make sense to include other parameters after a catchall parameter. The matching process is identical, regardless of whether you use one asterisk or two. The difference between using one or two asterisks is apparent when you use the route template to generate URLs (which you will see later in this chapter). If you use one asterisk, path separators within the generated URL will be URL-encoded—that is, they will be rendered as `%2F`. So for example, the URL /City/London/2022/4/18 will be rendered as /City%2FLondon%2F2022%2F4%2F18. When you use two asterisks, the encoding is decoded, or *round-tripped*, and the generated URL will include literal path separators: /City/London/2022/4/18.

4.2.5 Route constraints

As you should have gathered by now, route parameters are analogous to the method parameters you encounter in most programming languages. They are placeholders for variable data provided to parts of the application at run time. Therefore, it is inevitable that you will want to be able to perform some kind of processing on these inputs more often than not. You may want to construct a .NET `DateTime` object with the route values passed in to the arrival date example, so you can work with it in .NET code: